BÀI THỰC HÀNH CHƯƠNG 3

I. Quản lý người dùng và phân quyền với vai trò quản trị hệ thống

A. Tạo tài khoản Superuser

Phần này sẽ thực hành tạo một tài khoản **superuser** để đăng nhập vào trang quản trị (**Admin**) của hệ thống. Chức năng này sẽ được sử dụng trong các bài thực hành tiếp theo mà chỉ có người dùng có tài khoản quản trị mới có thể thực hiện.

1. Nhập lệnh sau vào **terminal** để tạo một **superuser**:

```
python manage.py createsuperuser
```

   Sau khi nhập lệnh này, hệ thống sẽ yêu cầu nhập các thông tin về tài khoản quản trị, gồm tên đăng nhập, email (tùy chọn) và mật khẩu.

2. Nhập tên đăng nhập và email theo từng chỉ dẫn trên dòng lệnh. Trong trường hợp ứng dụng **Bookr**, nhập tên tài khoản là **bookradmin** và email là **bookradmin.example.com**. Nhấn **Enter** sau mỗi thông tin nhập vào.

```
Username (leave blank to use 'django'): bookradmin
Email address: bookradmin@example.com
Password:
```

3. Tiếp theo, nhập mật khẩu (**mạnh**) và nhập lại mật khẩu **lần 2** để xác nhận.

```
Password:
Password (again)
```

   Sau khi hoàn tất việc nhập mật khẩu, sẽ nhận được dòng thông báo sau:

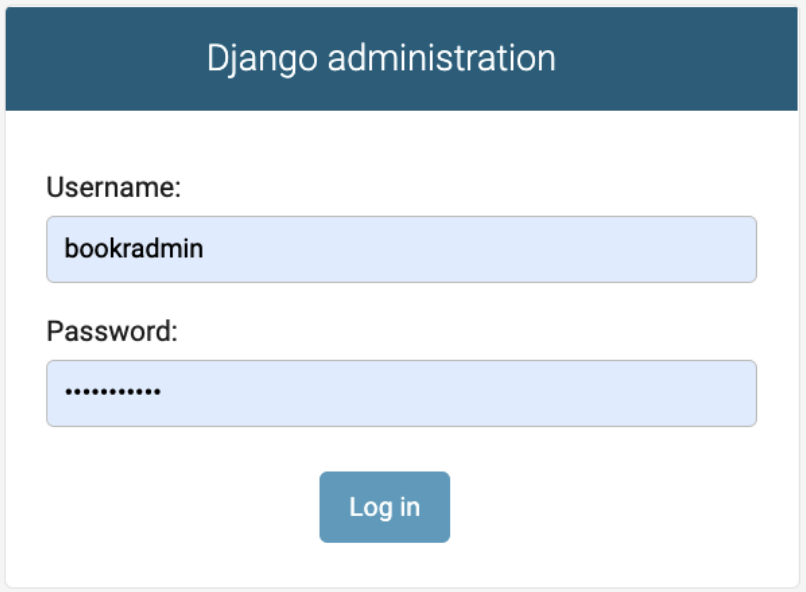
```
Superuser created successfully.
```

   Mật khẩu **mạnh** trong Django, cần thỏa mãn các điều kiện sau:

   - Không nằm trong nhóm 20.000 mật khẩu phổ biến;
   - Có chiều dài tối thiểu là 8 ký tự;
   - Không chỉ có số và chữ mà phải gồm cả các ký tự đặc biệt khác;
   - Không chứa thông tin về tên đăng nhập, họ, tên hay địa chỉ email của người dùng.

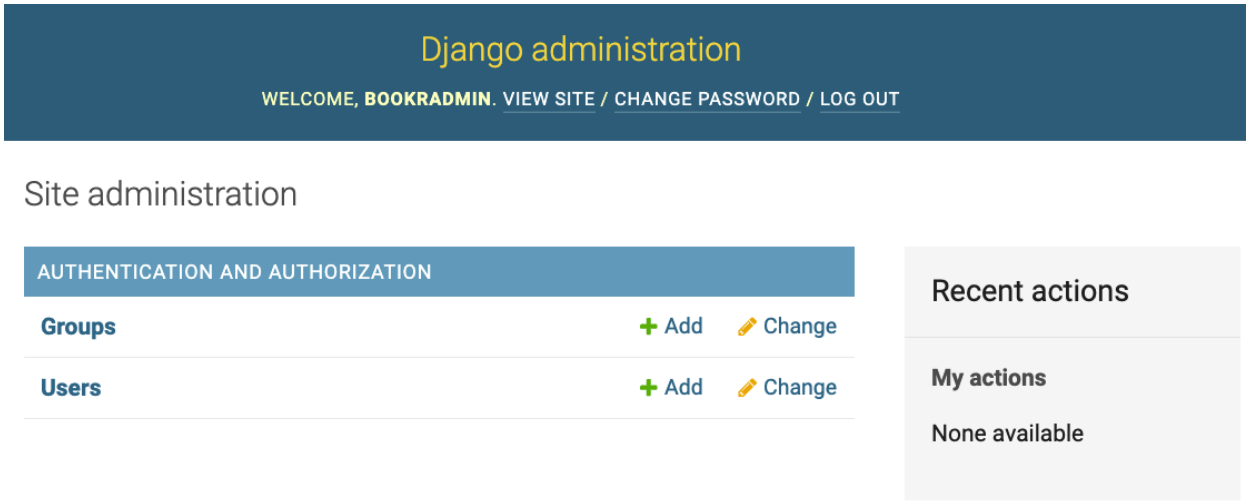
   Trong trường hợp mật khẩu không đủ mạnh, hệ thống sẽ yêu cầu cung cấp mật khẩu khác.

4. Truy cập đến trang http://127.0.0.1:8000/admin và đăng nhập bằng tài khoản **superuser** vừa tạo. Giao diện trang đăng nhập như Hình I-1.



Hình I-1. Trang đăng nhập để quản trị hệ thống

Sau khi đăng nhập vào hệ thống với tài khoản superuser, giao diện quản trị hệ thống (App Admin) của Django có dạng như Hình I-2.



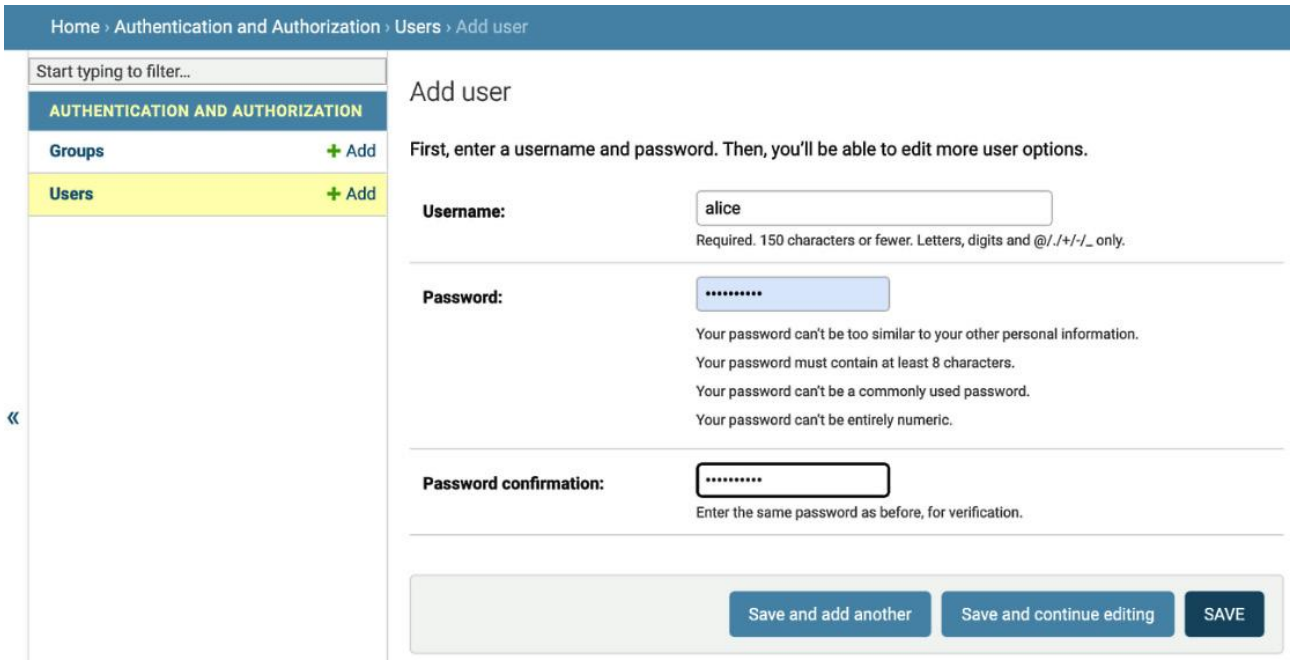
Hình I-2. Giao diện trang Quản trị hệ thống của Django

Khi khởi tạo, App Admin gọi một phương thức là **autodiscover()** để xác định xem có các app khác đã được cài đặt vào hệ thống không. Nếu có thì sẽ import các lớp quản trị của từng Model. Trong trường hợp của dự án **Bookr** hiện tại, App Admin đã tìm ra module **django.contrib.auth.admin** với 2 model là **Groups** và **Users**. Vì thế, module này được import và kết quả như hình trên.

Các phần tiếp theo sẽ thực hành một số thao tác liên quan đến quản lý người dùng, nhóm và phân quyền.

B. Thêm mới tài khoản người dùng

1. Từ giao diện quản trị (Hình I-2), bấm chuột vào nút **Add** của dòng **Users** để thêm mới người dùng, giao diện trang **Add user** xuất hiện sẽ như Hình I-3.



Hình I-3. Trang Add user

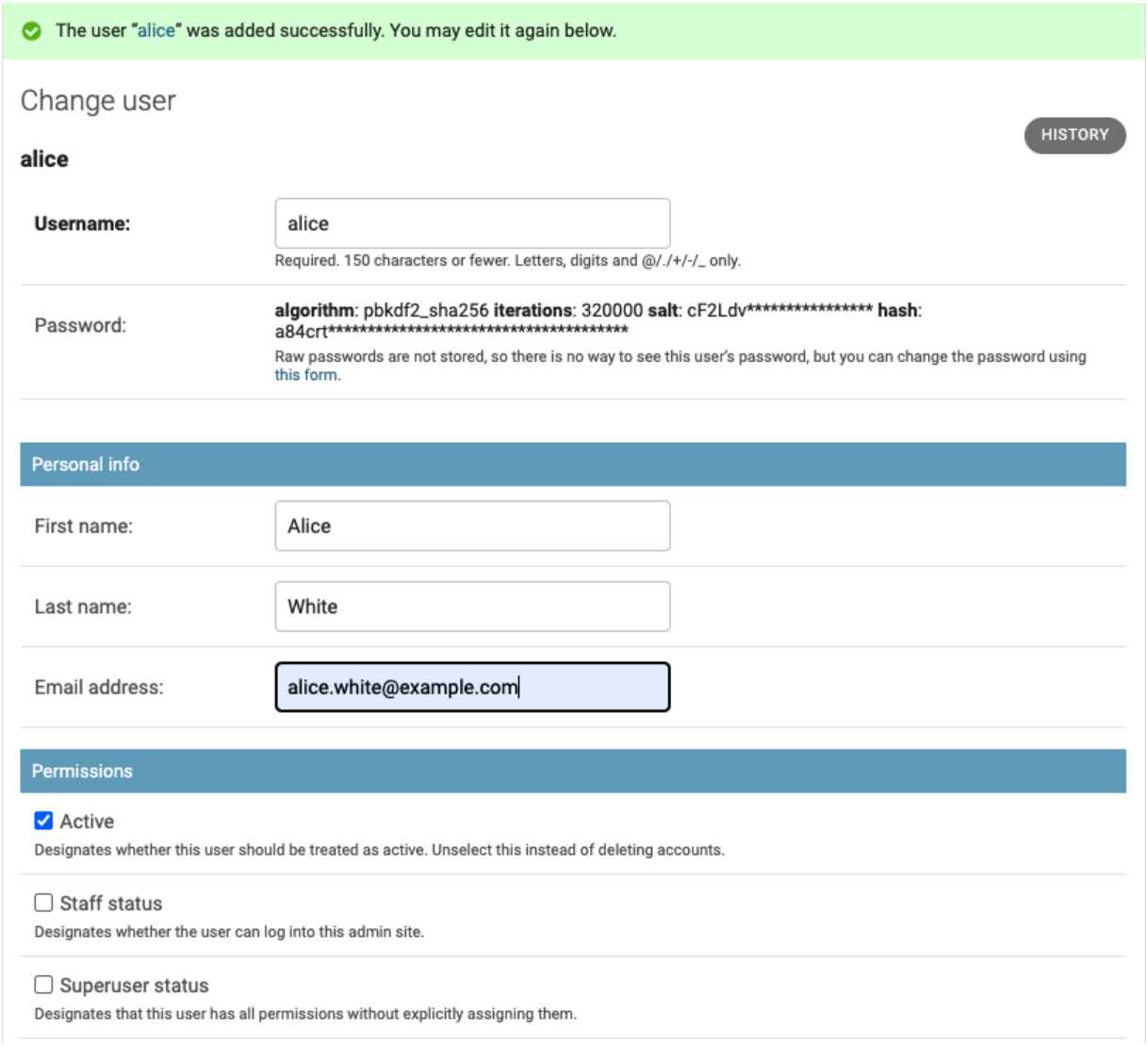
2. Điền tài khoản người dùng và mật khẩu **mạnh** (2 lần).

   Phần cuối của trang có 3 nút:

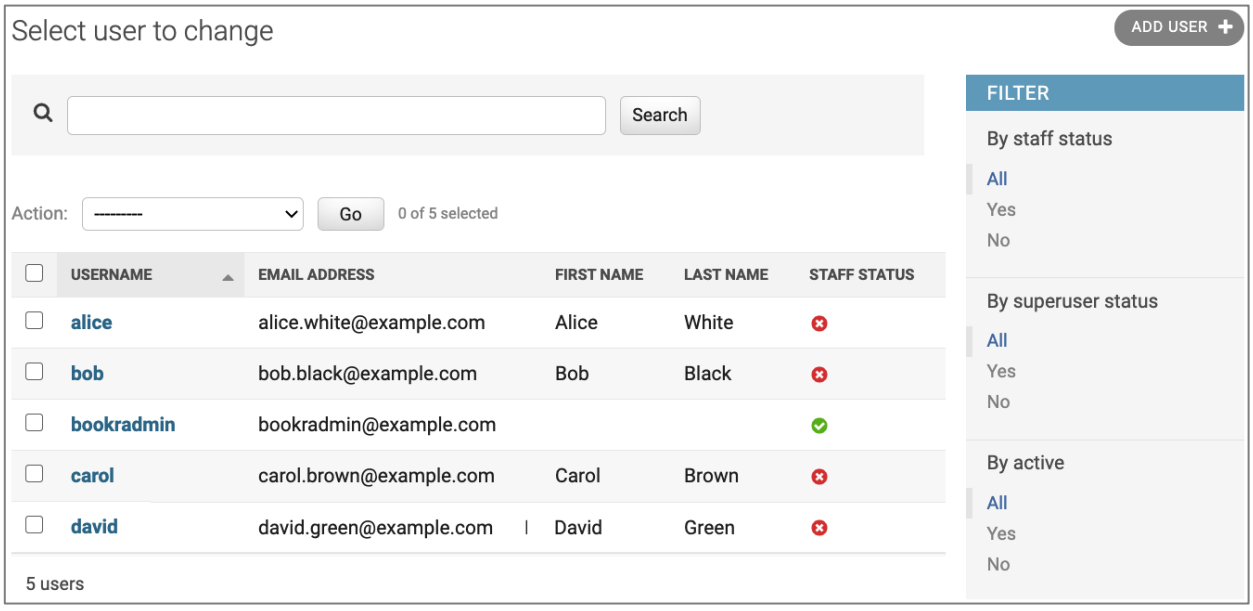
   - ***Save and add another**: Tạo người dùng và mở lại trang **Add user** để thêm người dùng tiếp theo.*
   - ***Save and continue editing**: Tạo người dùng và mở trang **Change user** để nhập thêm các thông tin bổ sung cho người dùng như Họ, Tên,... như trong Hình I-4. Lưu ý là trang này không cho nhập mật khẩu trực tiếp và cần mở đến trang riêng như link đính kèm trong mục này.*
   - ***SAVE**: Tạo người dùng và chuyển đến trang danh sách người dùng (Tiêu đề trang là **Select user to change**) như trong Hình I-5.*

   Chọn mục **Save and continue editing**, sau đó nhập các trường thông tin như trong form ở Hình I-4 và lưu lại thông tin.

3. Tại trang **Select user to change**, bấm chuột vào nút **Add User** và thực hiện thêm tiếp các tài khoản ***bob***, ***carol*** và ***david*** như trong Hình I-5.



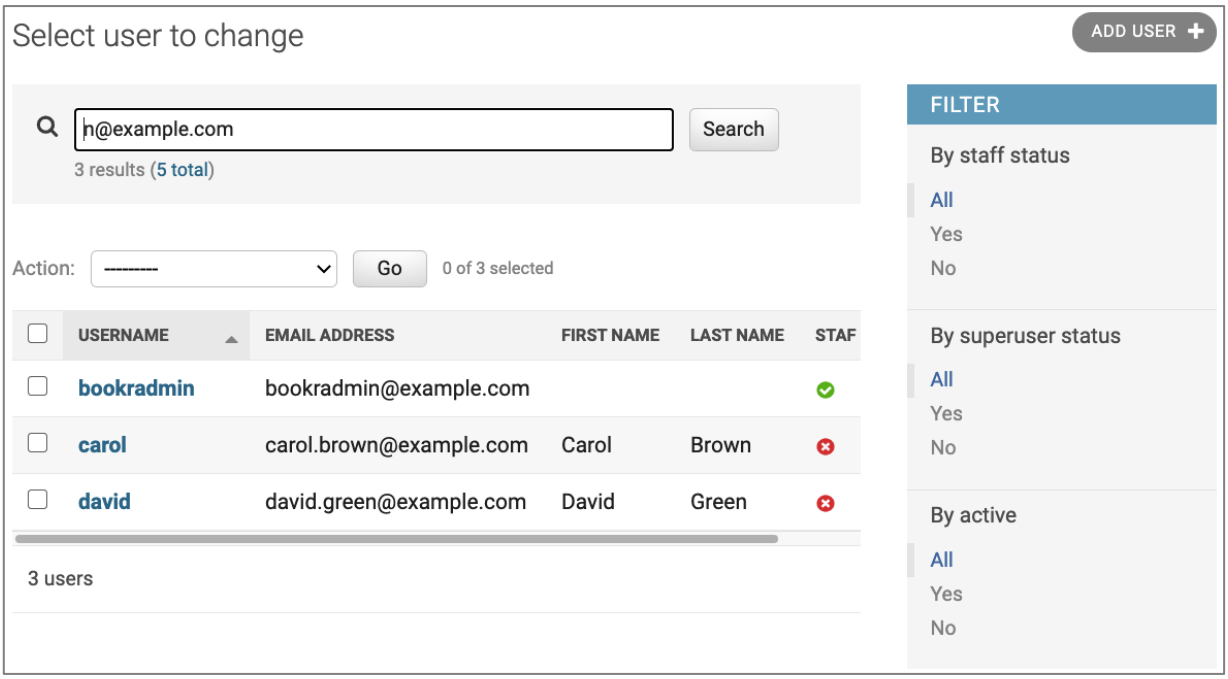
Hình I-4. Trang Change user



Hình I-5. Trang Select user to change

C. Truy xuất và cập nhật thông tin người dùng

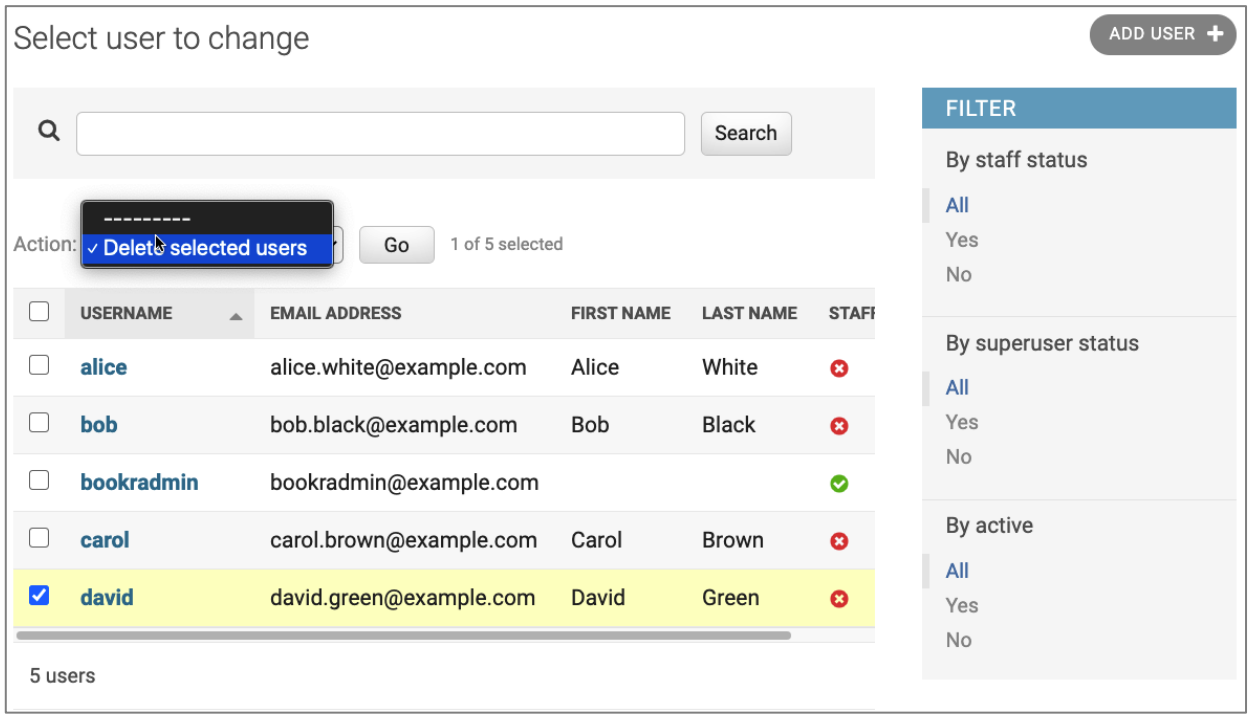
1. Tại trang **Select user to change** có thể tìm kiếm hoặc lọc người dùng thông qua hộp nhập **Search** và mục **FILTER** ở phía bên phải trang. Hình I-6 minh họa kết quả tìm kiếm khi nhập ***n@example.com*** vào hộp nhập **Search**.
2. Để xem và chỉnh sửa thông tin chi tiết của người dùng, bấm chuột vào tên của tài khoản người dùng tương ứng trong cột **USERNAME**. Khi đó sẽ mở trang **Change user** như trong Hình I-4.



Hình I-6. Kết quả tìm kiếm người dùng sau khi nhập nội dung tìm n@example.com

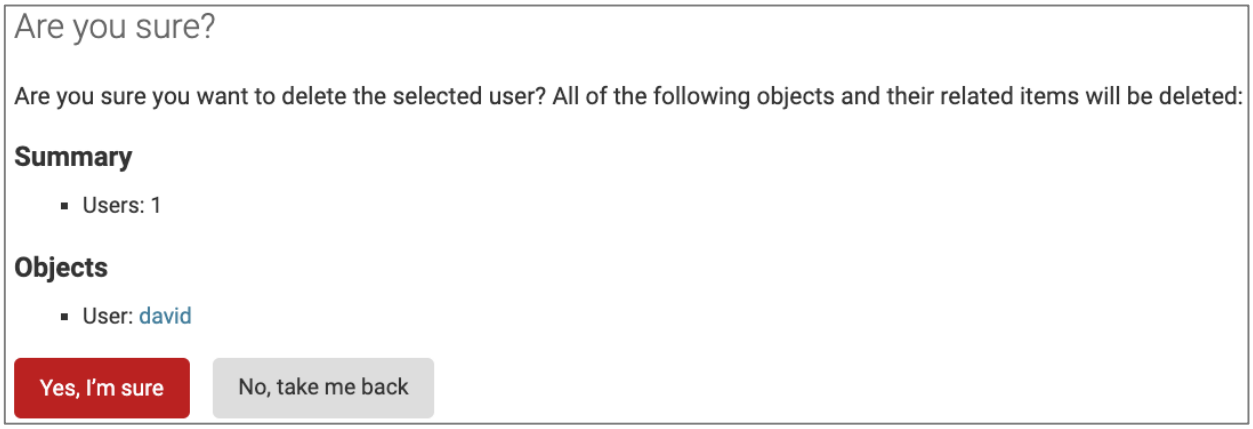
D. Xóa người dùng

1. Tại trang **Select user to change**, để xóa một hoặc một số người dùng, cần bấm chuột vào hộp chọn phía bên trái mỗi dòng để chọn người dùng tương ứng, sau đó chọn mục **Delete selected users** trong mục **Action** và nhấn nút **Go** như tại Hình I-7.



Hình I-7. Chọn người dùng để xóa

2. Xuất hiện trang xác nhận xóa người dùng cùng với các thông tin liên quan như trong Hình I-8. Có thể bấm nút **Yes, I'm sure** để xóa hoặc bấm nút **No, take me back** để hủy thao tác và quay trở lại trang **Select user to change.**



Hình I-8. Trang xác nhận xóa người dùng.

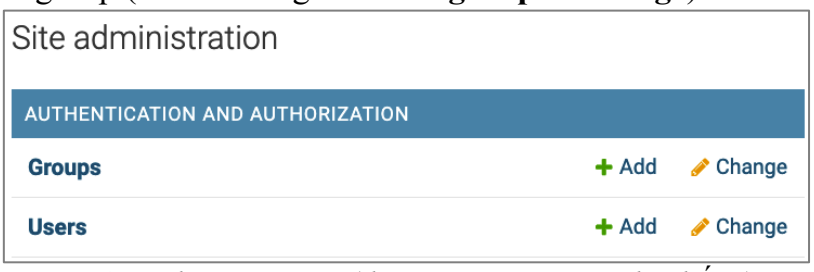
Bấm nút **Yes, I'm sure** để xác nhận xóa và người dùng ***David*** sẽ bị xóa vĩnh viễn.

Lưu ý: *Có thể khóa tạm thời người dùng thay vì xóa hoàn toàn bằng cách chọn mục chỉnh sửa thông tin chi tiết của người dùng như ở bài thực hành trước và bỏ chọn mục* ***Active*** *trong phần* ***Permissions*** *của trang* ***Change user*** *(như Hình I-4).*

E. Thêm nhóm người dùng và phân quyền người dùng

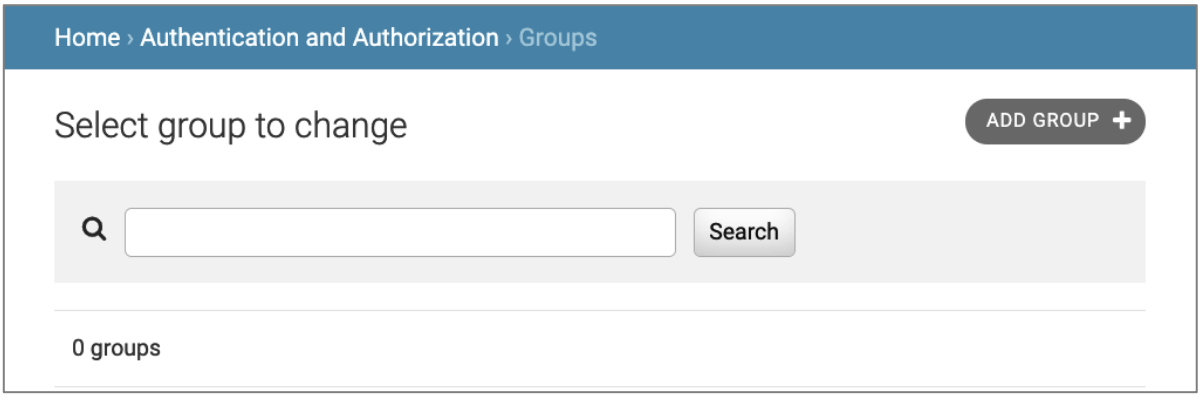
Phần này sẽ phân quyền quản trị cho một người dùng của dự án **Bookr** là ***Carol***. Để thực hiện việc này, trước tiên cần tạo một nhóm và phân quyền cho nhóm này, sau đó thêm người dùng ***Carol*** vào nhóm.

1. Vào trang quản trị tại địa chỉ http://127.0.0.1:8000/admin/ và đăng nhập với tài khoản ***Bookradmin*** đã thêm ở bài thực hành trước.
2. Tại phần **AUTHENTICATION AND AUTHORIZATION** bấm vào chữ **Groups** để mở trang danh sách group (tiêu đề trang là **Select group to change**)



Hình I-9. Mục Site administration (thuộc trang Quản trị hệ thống) của Django

3. Tại trang **Select group to change**, bấm vào nút **ADD GROUP** để mở trang **Add group**.



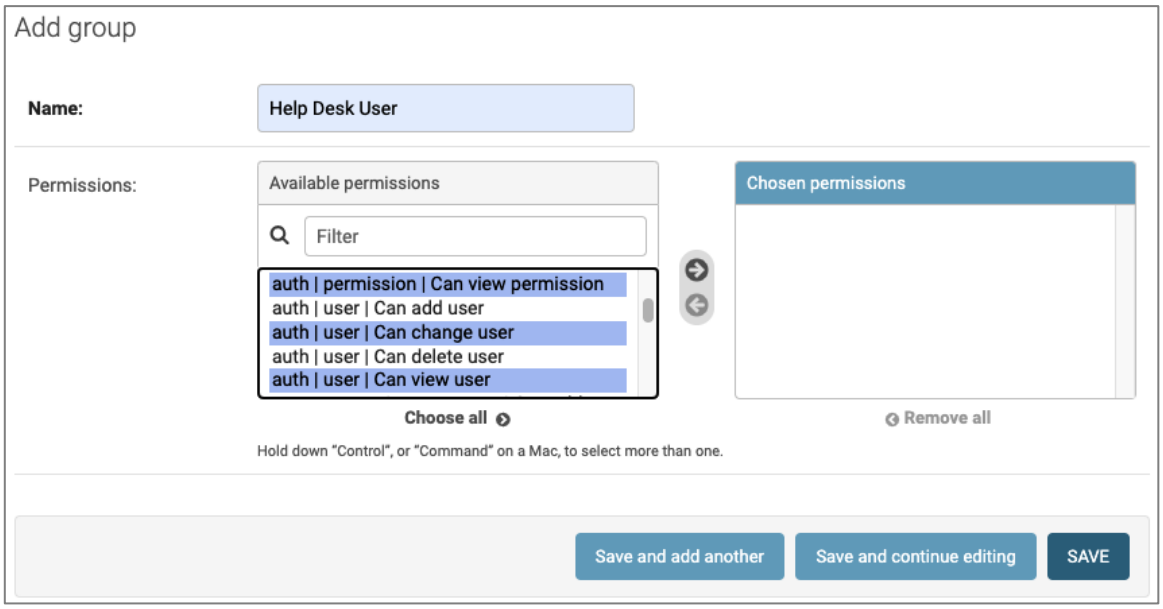
Hình I-10. Trang Select group to change

Lưu ý: *Ta cũng có thể mở trang **Add group** từ trang Quản trị hệ thống trang quản trị bằng cách chọn mục **Add** ở dòng **Groups**.*

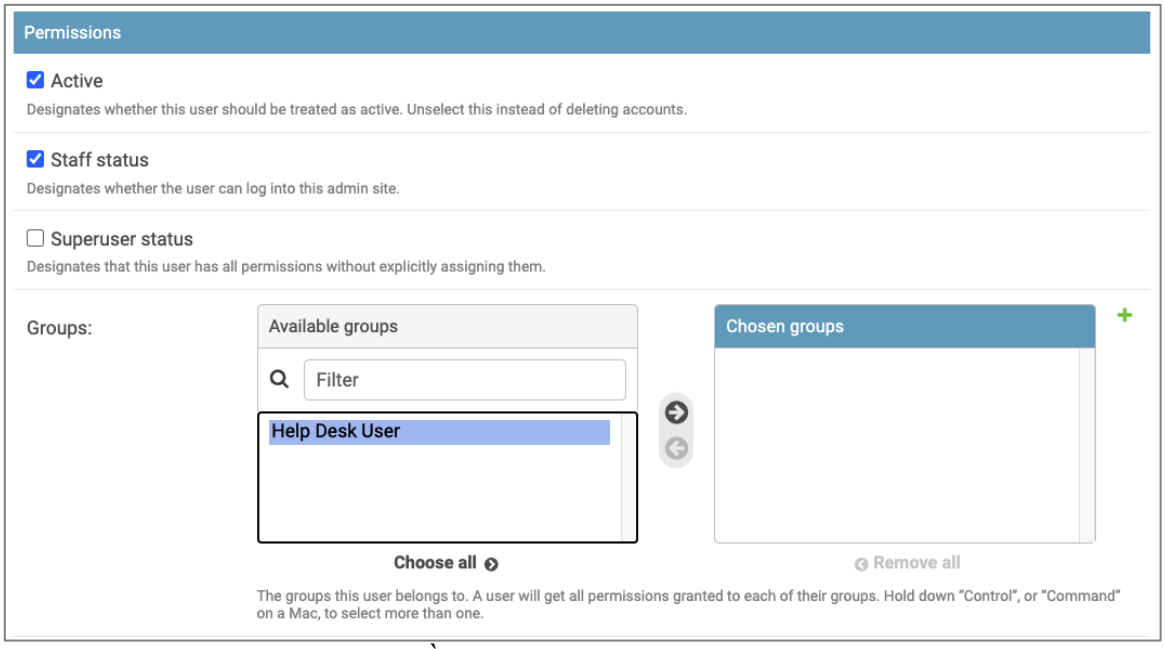
4. Tại trang **Add group** như Hình I-11, nhập tên nhóm là ***Help Desk User*** và chọn các quyền:
   - **Can view log entry**
   - **Can view permission**
   - **Can change user**
   - **Can view user**

   Để chọn các quyền này, chọn các mục tương ứng trong hộp danh sách bên trái phần **Permissions** và bấm nút mũi tên sang phải (→). Có thể chọn nhiều quyền cùng lúc bằng cách giữ phím **Ctrl** khi chọn các mục.

   Sau khi nhập và chọn xong các thông tin thì nhấn nút **SAVE** để hoàn tất việc thêm nhóm.
5. Mở trang **Select user to change** và chọn người dùng ***Carol*** từ trong danh sách để mở trang **Change user** cho tài khoản ***Carol***.
6. Tại trang **Change user**, cuộn xuống mục **Permissions** và chọn mục **Staff status** như trong Hình I-12. Tùy chọn này cho phép người dùng ***Carol*** có thể đăng nhập vào App Admin.

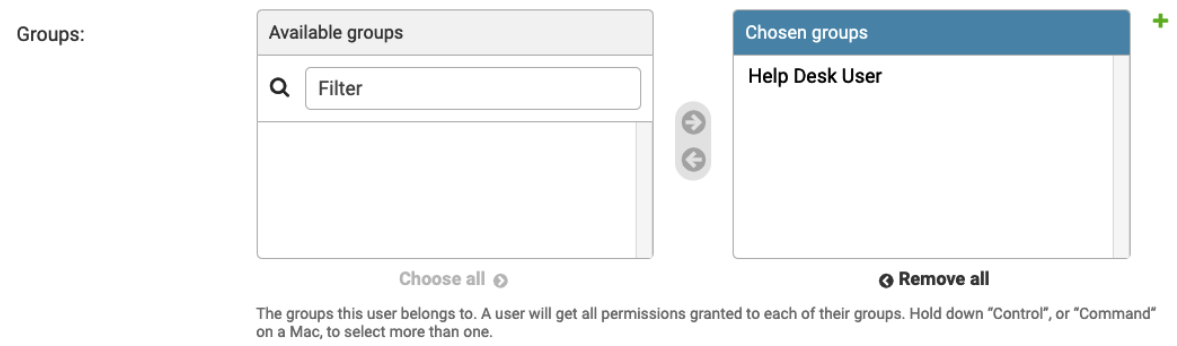


Hình I-11. Trang Add group



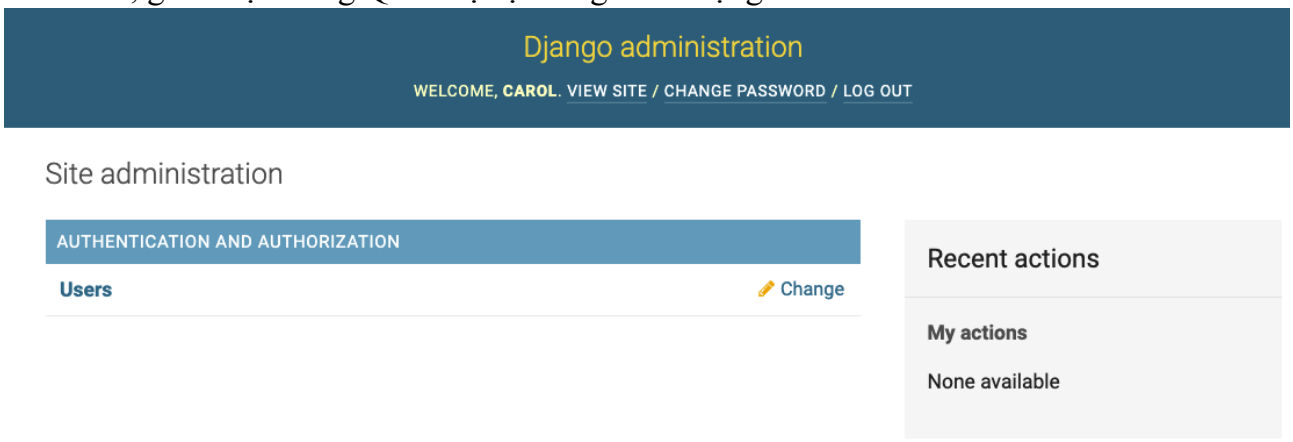
Hình I-12. Phần Permissions của trang Change user

7. Thêm người dùng ***Carol*** vào nhóm ***Help Desk User*** đã tạo ở trên bằng cách chọn nhóm này ở mục **Groups** và bấm vào nút mũi tên sang phải (➔). Kết quả sẽ như Hình I-13.



Hình I-13. Nhóm Help Desk User đã được chọn

8. Sau khi lưu các sửa đổi cho tài khoản người dùng ***Carol***, có thể kiểm tra quyền của người dùng này bằng cách đăng xuất khỏi tài khoản ***Bookradmin*** hiện tại và đăng nhập vào tài khoản ***Carol*** từ trang đăng nhập Admin. Sau khi đăng nhập thành công bằng tài khoản ***Carol***, giao diện trang Quản trị hệ thống sẽ có dạng như Hình I-14.



Hình I-14. Trang Quản trị hệ thống Bookr của tài khoản Carol

II. Đăng ký Model và quản lý dữ liệu các Model với App Admin

Phần này thực hiện đăng ký các Model của App **Reviews** với App **Admin** để từ đó có thể thực hiện các thao tác quản lý dữ liệu của App **Reviews** trên giao diện web của App **Admin**.

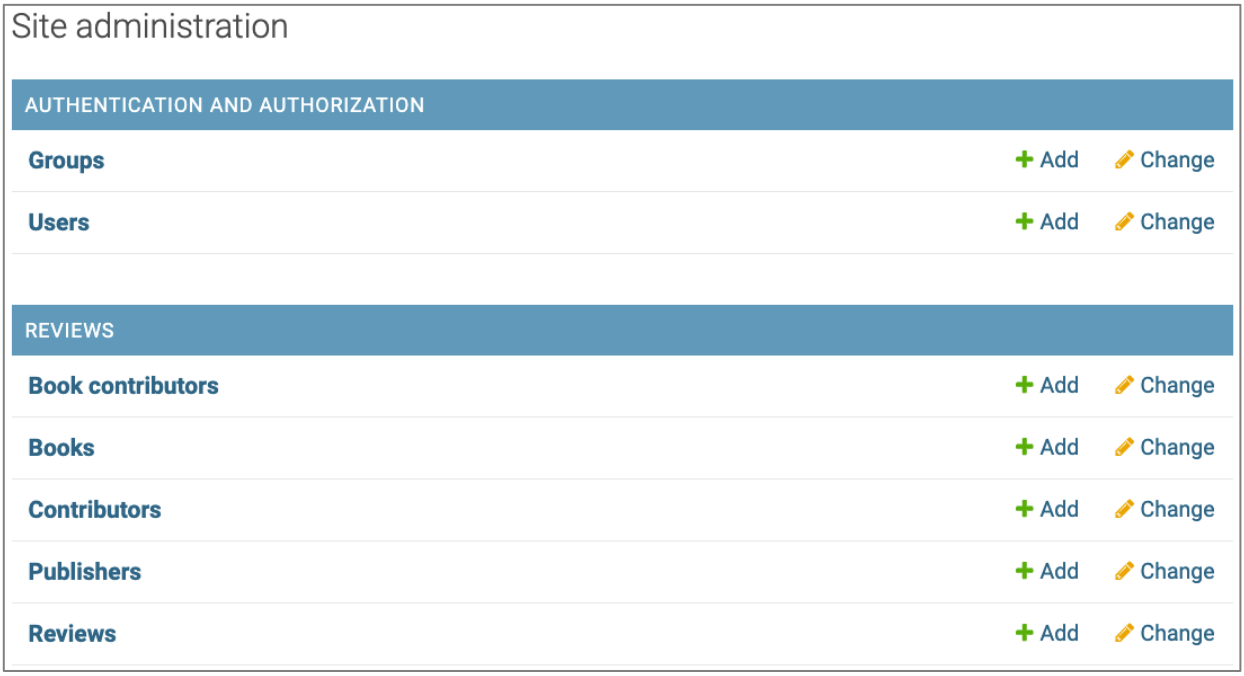
A. Đăng ký Model với App Admin

1. Thực hiện đăng ký các Model bằng cách mở tập tin **reviews/admin.py** và thêm vào các dùng mã lệnh sau:

```
from django.contrib import admin
from reviews.models import (Publisher, Contributor, Book,
BookContributor, Review)

# Register your models here.
admin.site.register(Publisher)
admin.site.register(Contributor)
admin.site.register(Book)
admin.site.register(BookContributor)
admin.site.register(Review)
```

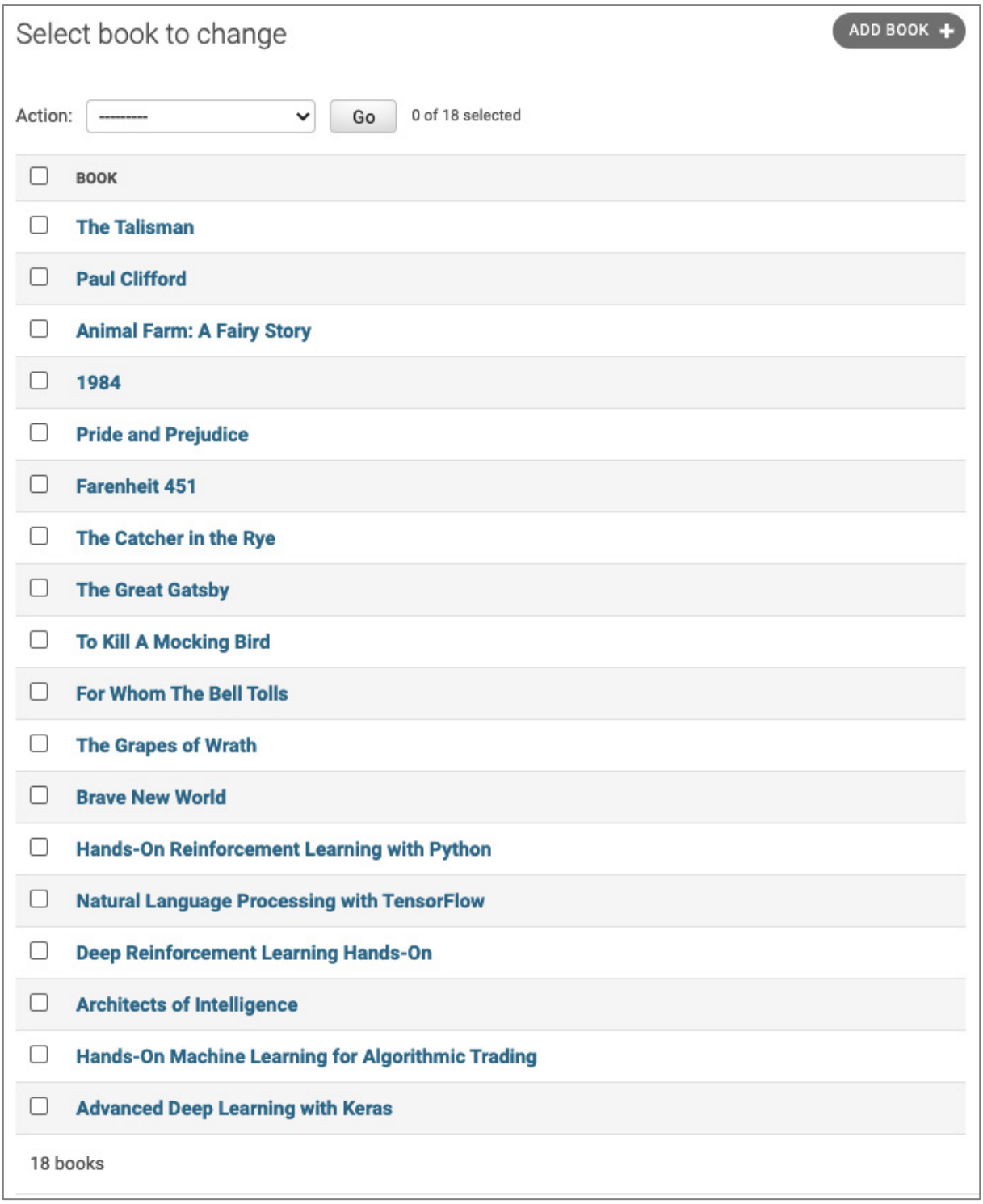
2. Tải lại trang http://127.0.0.1:8000/admin/ trên trình duyệt web, giao diện trang Quản trị hệ thống sẽ thay đổi và có dạng như Hình II-1.



Hình II-1. Giao diện trang Quản trị hệ thống sau khi đăng ký thêm 5 Model

B. Quản lý dữ liệu Model với App Admin

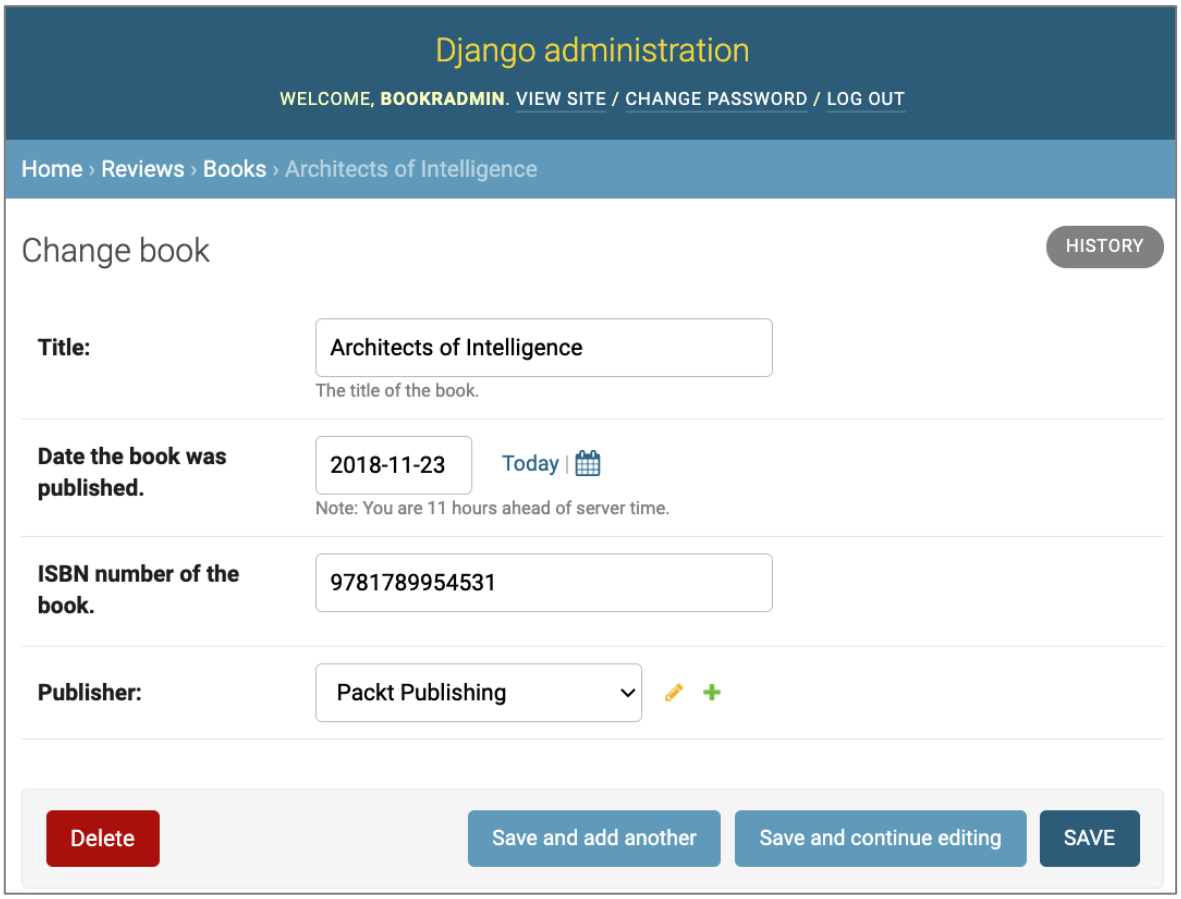
1. Từ trang Quản trị hệ thống tại Hình II-1, mở trang quản lý sách bằng cách bấm chuột vào chữ **Books** (hoặc nút **Change**) trên dòng **Books** của phần **REVIEWS**. Trang quản lý sách (tiêu đề là **Select book to change**) sẽ mở ra như trong Hình II-2. Trang này có địa chỉ URL như sau http://127.0.0.1:8000/admin/reviews/book.



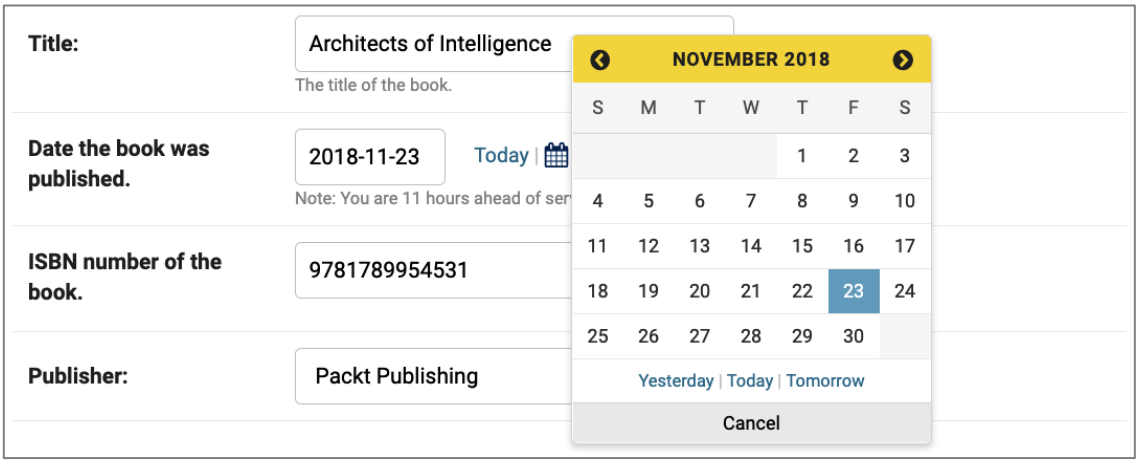
Hình II-2. Trang Select book to change

2. Bấm chuột vào tiêu đề 1 cuốn sách như ***Architects of Intelligence*** sẽ mở ra trang **Change book** (có địa chỉ http://127.0.0.1:8000/admin/reviews/book/3/change/) như Hình II-3. Ta có thể chỉnh sửa lại thông tin về cuốn sách tại đây.
   Đối với dữ liệu kiểu Ngày tháng (**Date**) như mục **Date the book was published**., ta có thể chọn ngày tháng từ hộp chọn ngày tháng (như Hình II-4).

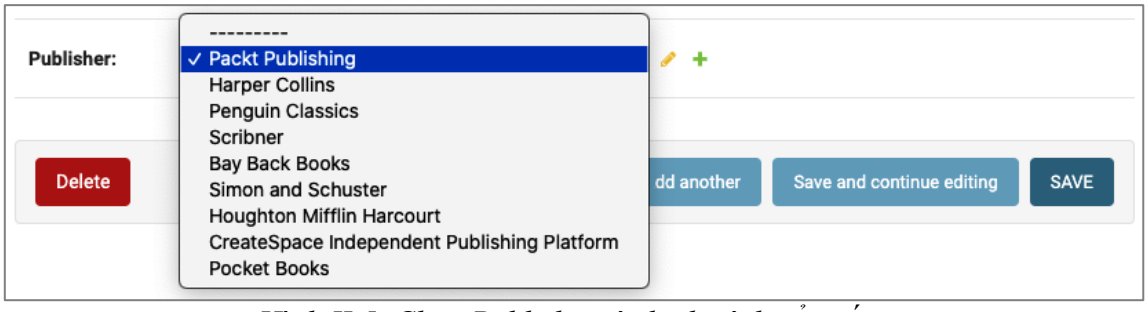
   Đối với mục có liên kết đến bảng khác như mục **Publisher**, có thể chọn mục liên kết tương ứng từ danh sách sổ xuống. Các mục trong danh sách này được lấy từ model **Publisher**.



Hình II-3. Trang Change book



Hình II-4. Hộp chọn ngày tháng

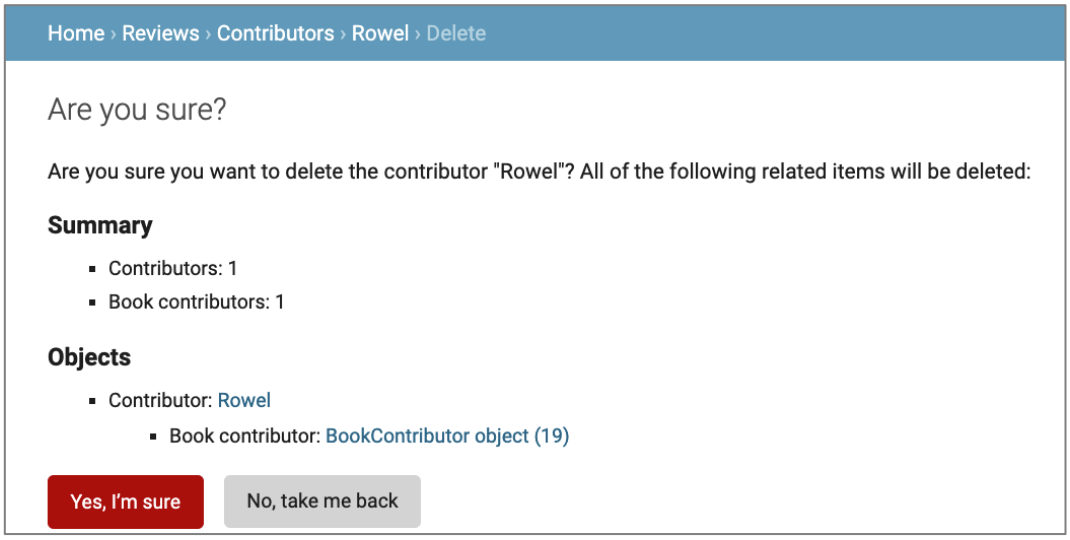


Hình II-5. Chọn Publisher từ danh sách sổ xuống

C. Cách thức xóa dữ liệu có liên quan đến khóa ngoại

Trong bài thực hành ở Chương 2, tất cả các khóa ngoại của app **Reviews** được định nghĩa **on_delete=CASCADE**. Điều này có nghĩa là khi nhà quản trị xóa một bản ghi có khóa ngoại tham chiếu đến thì tất cả các bản ghi có khóa ngoại tham chiếu đến đó sẽ bị xóa cùng. Ví dụ như nếu xóa một nhà xuất bản thì tất cả các cuốn sách của nhà xuất bản đó sẽ bị xóa. Để thay đổi điều này cần điều chỉnh lại hành vi của Model khi xóa các bản ghi thành **PROTECT**.

1. Vào trang quản lý Contributor bằng link http://127.0.0.1:8000/admin/reviews/contributor/ và chọn để xóa một contributor là tác giả của một cuốn sách nào đó, ví dụ như ***Rowel***.
2. Bấm vào nút **Delete** nhưng **KHÔNG** bấm vào nút **Yes, I'm sure** trong hộp thoại xác nhận sau:



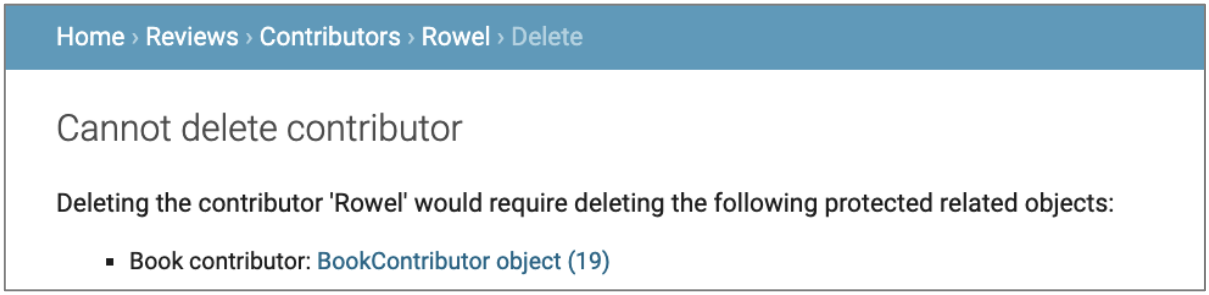
Hình II-6. Hộp thoại xác nhận xóa các bản ghi có quan hệ với bản ghi được xóa

Theo hộp thoại này, có 19 bản ghi **BookContributor** có thể sẽ bị xóa cùng nếu xóa contributor được chọn. Đây là do thiết đặt **on_delete=CASCADE** của mối quan hệ giữa model **Book** và **Contributor**.

3. Điều chỉnh lại hành vi **on_delete** trên bằng cách mở tập tin **reviews/models.py** và sửa phần định nghĩa thuộc tính **Contributor** của lớp **BookContributor** như sau và lưu tập tin lại:

```
contributor = models.ForeignKey(Contributor,
    on_delete=models.PROTECT)
```

4. Bây giờ thực hiện lại thao tác xóa contributor như ở bước 1, một thông báo khác với hộp thoại xác nhận ở bước 2 sẽ hiện ra, dạng như Hình II-7.



Hình II-7. Lỗi không thể xóa vì ràng buộc bảo vệ khóa ngoại

5. **Khôi phục lại** đoạn mã lệnh đã sửa đổi ở bước 3 như ban đầu.

III. Tùy chỉnh giao diện các trang của App Admin

A. Truy xuất thông tin tùy chỉnh ở cấp toàn cục từ đối tượng AdminSite

1. Khởi chạy máy chủ web bằng lệnh:

```
python manage.py runserver
```

2. Khởi chạy giao diện dòng lệnh của **shell** Django bằng lệnh

```
python manage.py shell
```

3. **Import** gói **django.contrib.admin**

```
>>> from django.contrib import admin
```

4. Truy xuất thông thông tin

```
>>> admin.site.site_title
'Django site admin'
>>> admin.site.site_header
'Django administration'
>>> admin.site.index_title
'Site administration'
```

B. Sử dụng tính kế thừa để thay đổi các tùy chỉnh toàn cục

Phần này sẽ điều chỉnh lại mã nguồn của tập tin **review/admin.py** để tạo lớp mới kế thừa từ lớp **AdminSite** và dùng lớp mới này để thay đổi các tùy chỉnh cho các trang của App **Admin** thay cho việc dùng đối tượng site từ gói **django.contrib.admin** như đã làm ở bài thực hành phần II.A.

1. Điều chỉnh lại mã nguồn của tập tin **review/admin.py** như sau:

```
from django.contrib.admin import AdminSite
from reviews.models import (Publisher, Contributor, Book,
BookContributor, Review)

class BookrAdminSite(AdminSite):
    title_header = 'Bookr Admin'
    site_header = 'Bookr administration'
    index_title = 'Bookr site admin'

admin_site = BookrAdminSite(name='bookr')

# Register your models here.
admin_site.register(Publisher)
admin_site.register(Contributor)
admin_site.register(Book)
admin_site.register(BookContributor)
admin_site.register(Review)
```

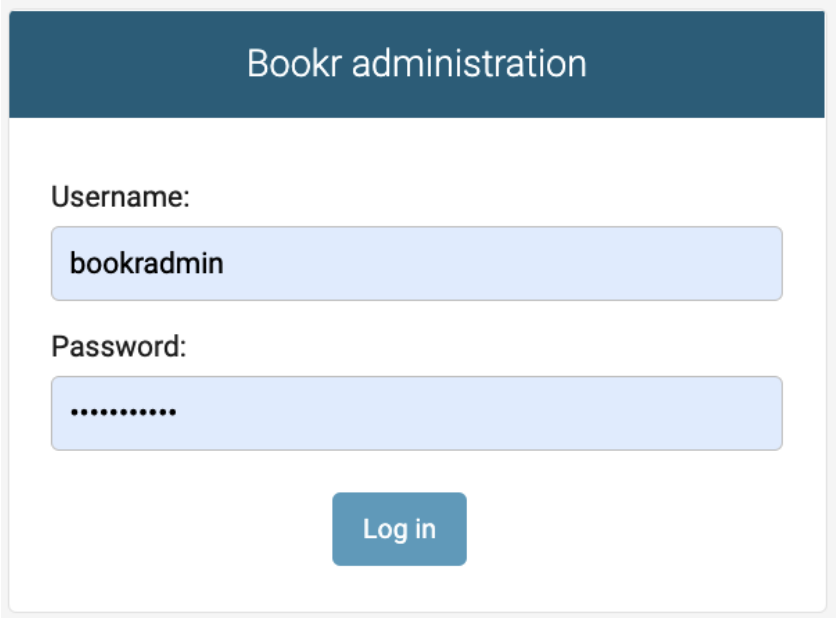
Trong bản điều chỉnh này, đã định nghĩa một lớp mới là **BookrAdminSite** kế thừa từ lớp **AdminSite**. Chúng ta thay đổi một số tùy chỉnh toàn cục cho App Admin bằng cách gán các giá trị cho các thuộc tính **title_header**, **site_header** và **index_title**. Sau đó, tạo đối tượng **admin_site** của lớp **BookrAdminSite** và sử dụng đối tượng này để đăng ký các lớp Model.

2. Điều chỉnh lại ánh xạ URL đến App Admin trong tập tin **bookr/urls.py** để trỏ đến đối tượng **admin_site** mới này như sau:

```
from reviews.admin import admin_site
import reviews.views
from django.urls import include, path

urlpatterns = [
    path("admin/", admin_site.urls),
    path("book-search", reviews.views.book_search),
    path("", include("reviews.urls")),
]
```

3. Mở lại trang http://127.0.0.1:8000/admin/ và gia diện trang đăng nhập sẽ thay đổi thành như hình sau:



Hình III-1. Giao diện trang đăng nhập đã được điều chỉnh

C. Sử dụng tính kế thừa để thay đổi các tùy chỉnh toàn cục (cách 2)

Việc sử dụng cách tạo lớp **BookrAdminSite** như ở bài thực hành trên đã tạo ra giao diện thứ 2 cho App Admin, điều này có nghĩa là hệ thống có 2 giao diện làm việc song song: một giao diện từ đối tượng **django.contrib.admin.site** và một giao diện tùy chỉnh từ **admin_site**. Điều này có thể gây bối rối cho các nhóm tham gia phát triển ứng dụng. Để giải quyết, từ phiên bản **Django 2.1** có thể thực hiện tích hợp giao diện tùy chỉnh theo cách không cần phải thay đổi thiết đặt hiện có.

1. Do lớp **BookrAdminSite** tùy chỉnh giao diện ở cấp độ dự án nên mã nguồn của lớp này không nên đặt trong thư mục của App **Reviews** mà nên đặt ở thư mục chung của dự án. Để thực hiện việc này, ta tạo một tập tin **admin.py** mới ở **thư mục gốc** của dự án **Bookr** và chuyển mã nguồn định nghĩa lớp **BookAdminSite** sau đây từ tập tin **reviews/admin.py** sang tập tin mới này:

```
from django.contrib import admin
class BookrAdminSite(admin.AdminSite):
    title_header = 'Bookr Admin'
```

```
    site_header = 'Bookr administration'
    index_title = 'Bookr site admin'
```

Ta cũng cần đổi lại **ánh xạ URL** của App Admin trong tập tin **bookr/urls.py** trở lại như trước khi sửa ở bài thực hành III.B (tức là: **`path('admin/', admin.site.urls)`** và lưu ý import module admin bằng dòng **`from django.contrib import admin`**) và định nghĩa một lớp **AdminConfig** tùy chỉnh mới tên là **ReviewsAdminConfig** như ở bước 2 sau đây.

2. Thêm một tập tin là **reviews/adminconfig.py** với nội dung như sau:

```
from django.contrib.admin.apps import AdminConfig
class ReviewsAdminConfig(AdminConfig):
    default_site = 'admin.BookrAdminSite'
```

3. Thay thế **django.contrib.admin** bằng **reviews.apps.ReviewsAdminConfig** trong biến **INSTALLED_APPS** của tập tin **bookr/settings.py**, kết quả là biến này sẽ như sau:

```
INSTALLED_APPS = [
    'reviews.adminconfig.ReviewsAdminConfig',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'reviews']
```

Với khai báo site mặc định trong lớp **ReviewsAdminConfig** thành lớp tùy chỉnh **BookrAdminSite**, ta không cần phải thay thế tham chiếu đến **admin.site** bằng đối tượng **admin_site** của lớp tùy chỉnh **AdminSite** nữa. Vì thế, ta có thể thay thế các phần mã có lời gọi đến đối tượng **admin_site** bằng **admin.site** như nguyên gốc ban đầu. Nội dung của tập tin **reviews/admin.py** khôi phục lại như sau:

```
from django.contrib import admin
from reviews.models import (Publisher, Contributor, Book,
BookContributor, Review)

# Register your models here.
admin.site.register(Publisher)
admin.site.register(Contributor)
admin.site.register(Book)
admin.site.register(BookContributor)
admin.site.register(Review)
```

D. Tùy chỉnh các lớp ModelAdmin

1. Hiện tại, giao diện trang quản lý sách (**Books**) còn khá sơ sài với chỉ một cột thông tin liệt kê tiêu đề của sách, cột thông tin này cũng chưa có tùy chọn để sắp xếp. Đồng thời chưa có mục chọn để lọc và tìm kiếm sách.

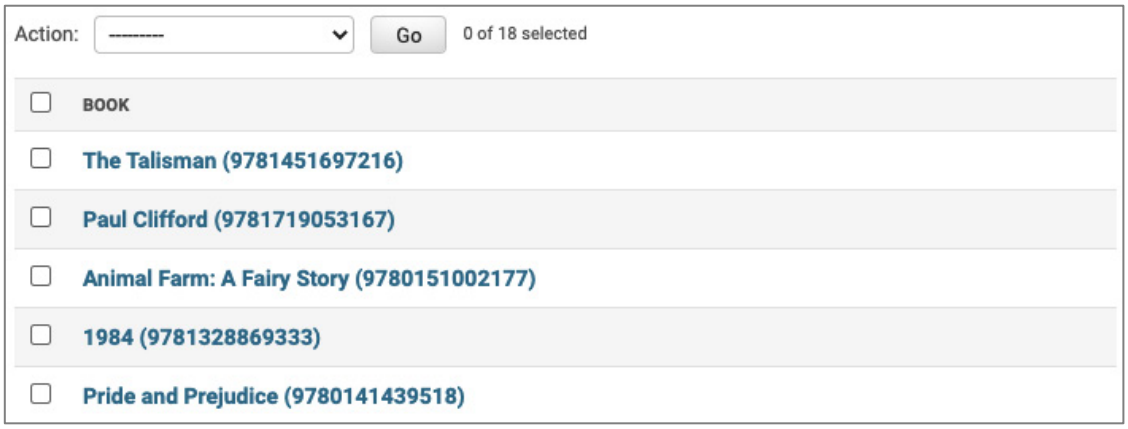
   Nhắc lại trong bài thực hành ở Chương 2, chúng ta đã định nghĩa phương thức **__str__()** trong các lớp Model **Publisher**, **Book** và **Contributor**. Trong lớp **Book**, phương thức **__str__()** trả về tiêu đề của cuốn sách. Nếu ta không định nghĩa phước thức này trong lớp

Book thì nó sẽ kế thừa từ lớp cha **django.db.models.Model**. Phương thức ở lớp cha này sẽ chỉ xuất ra kết quả ở dạng *Book object (**ID**)* với ID là giá trị khóa của đối tượng **Book**.

Ta có thể tùy chỉnh lại phần hiển thị thông tin của đối tượng **Book** trong trang quản lý sách để cung cấp thông tin từ nhiều trường. Ví dụ, ta có thể hiển thị cuối sách ở dạng **Tiêuđề(ISBN)** với đoạn mã sau đây trong phần định nghĩa phương thức **__str__()** của lớp Model **Book**:

```
class Book(models.Model):
    …
    def __str__(self):
        return "{} ({})".format(self.title, self.isbn)
```

Kết quả xuất ra ở trang quản lý sách sẽ thành như sau:



Hình III-2. Trang danh sách Book sau khi tùy chỉnh

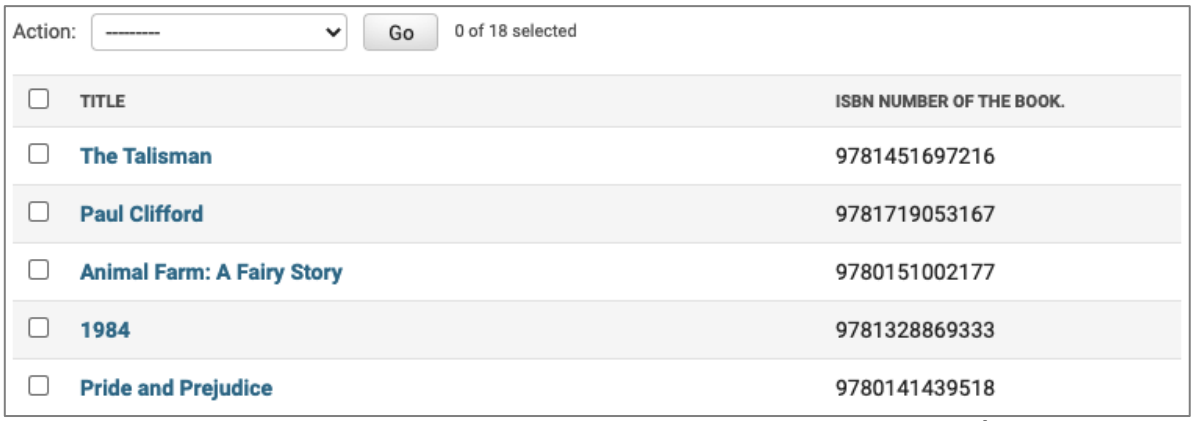
2. Ta có thể tùy chỉnh các cột thông tin hiển thị cho trang quản lý bằng cách thay đổi giá trị của thuộc tính **list_display** trong lớp **ModelAdmin** của module **django.contrib.admin**. Ví dụ, nếu ta muốn trang danh sách **Book** hiển thị 2 cột thông tin là **Tiêu đề (title)** và **ISBN**, ta có thể tạo một lớp **BookAdmin** kế thừa từ lớp **ModelAdmin** trong tập tin **reviews/admin.py** như sau:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ('title', 'isbn')
```

Để lớp **BookAdmin** này áp dụng cho trang quản lý của Model **Book**, ta cần tham chiếu đến lớp này khi đăng ký Model **Book** với App **Admin**. Dòng đăng ký model **Book** trong tập tin **reviews/admin.py** này sẽ điều chỉnh thành như sau:

```
admin.site.register(Book, BookAdmin)
```

Sau khi lưu nội dung điều chỉnh lại, giao diện của trang danh sách **Book** sẽ thay đổi thành như Hình III-3.



Hình III-3. Giao diện trang danh sách Book sau khi tùy chỉnh lần 2

Sinh viên đọc thêm tại các **trang 192-193** của **Giáo trình** để biết thêm về **4 loại giá trị** có thể gán cho thuộc tính **list_display**.

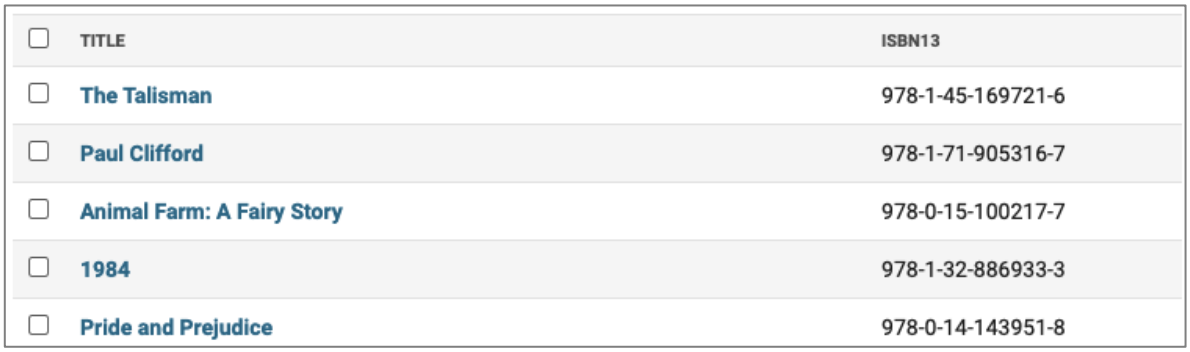
E. Thay đổi cách thức hiển thị các giá trị

Để thay đổi cách thức hiển thị giá trị ở các cột trong trang quản lý dữ liệu của **Model**, ta có thể sử dụng một số cách thức khác nhau. Phần này sẽ thực hành một số ví dụ:

1. Thay đổi định dạng giá trị của cột **ISBN** bằng cách định nghĩa một hàm là **isbn13()** và điều chỉnh tham chiếu trong thuộc tính **list_display**, như sau:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ('title', 'isbn13')
    def isbn13(self, obj):
        """ '9780316769174' => '978-0-31-676917-4' """
        return "{}-{}-{}-{}-{}".format(obj.isbn[0:3],
               obj.isbn[3:4], obj.isbn[4:6], obj.isbn[6:12],
               obj.isbn[12:13])
```

Kết quả, trang danh sách sẽ thành như sau:



Hình III-4. Trang danh sách Book sau khi điều chỉnh định dạng giá trị cột ISBN

2. Điều chỉnh tiêu đề cột **ISBN13** thành **ISBN-13** và thêm tính năng sắp xếp bằng cách sử dụng phương thức **admin.display()** như sau:

```
@admin.display(ordering='isbn', description='ISBN-13',
               empty_value='-/-')
def isbn13(self, obj):
    """ '9780316769174' => '978-0-31-676917-4' """
```

```
    return "{}-{}-{}-{}-{}".format(obj.isbn[0:3], obj.isbn[3:4],
                obj.isbn[4:6], obj.isbn[6:12], obj.isbn[12:13])
```

Trong hàm **admin.display**, tham số **ordering** để chọn cột thông tin trong CSDL sử dụng cho sắp xếp giá trị của cột, tham số **description** để đặt tiêu đề cột, và tham số **empty_value** để đặt nội dung hiển thị khi không có giá trị.

3. Thay đổi cách hiển thị giá trị kiểu **boolean** (luận lý) thành hình đại diện với đoạn mã sau:

```
@admin.display(boolean=True, description='Has ISBN', )
def has_isbn(self, obj):
    """ '9780316769174' => True """
    return bool(obj.isbn)
```

Tham số **boolean=True** của phương thức **admin.display** để thiết đặt cách thức hiển thị của giá trị kiểu **Boolean**.

Lưu ý: chỉnh lại giá trị thuộc tính **list_display** thành như sau:

```
list_display = ('title', 'isbn13', 'has_isbn')
```

Kết quả là trang danh sách **Book** sẽ thay đổi thành như Hình III-5.

TITLE	ISBN-13	HAS ISBN
To Kill A Mocking Bird	----	✗
Brave New World	978-0-06-085052-4	✓
Pride and Prejudice	978-0-14-143951-8	✓
The Grapes of Wrath	978-0-14-303943-3	✓
Animal Farm: A Fairy Story	978-0-15-100217-7	✓

*Hình III-5. Trang danh sách **Book** sau khi điều chỉnh tiêu đề cột ISBN13 và thêm cột Has ISBN*

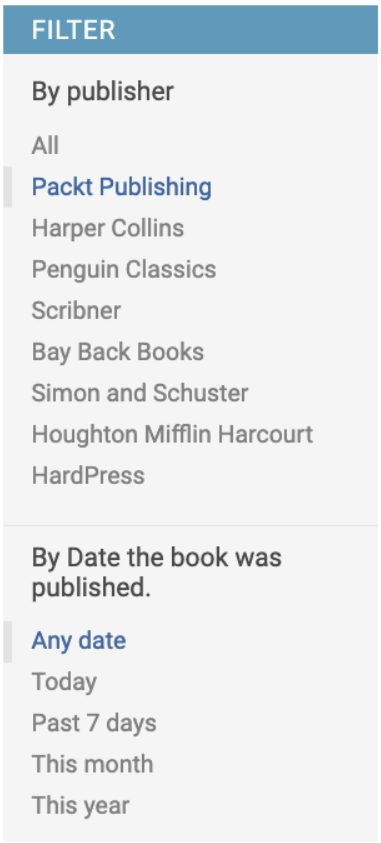
F. Thêm các bộ lọc

Khi cần quản lý một lượng lớn bản ghi thì việc lọc các bản ghi để giới hạn lại số lượng hiển thị sẽ rất có ích. Bài thực hành này sẽ thêm các bộ lọc theo những thông tin phổ biến của cuốn sách.

1. Chỉnh sửa lại tập tin **reviews/admin.py** để thêm các bộ lọc theo thuộc tính **publisher** và **publication_date** cho lớp **BookAdmin** như sau:

```
class BookAdmin(admin.ModelAdmin):
list_display = ('title', 'isbn13')
list_filter = ('publisher', 'publication_date')
```

2. Tải lại trang quản lý **Book**, ta sẽ thấy xuất hiện thêm phần **FILTER** chứa các bộ lọc ở cạnh trái cửa sổ trang web như Hình III-6.
   Bộ lọc về ngày xuất bản mới chỉ liệt kê được các khoảng thời gian gần đây như **trong ngày (Today), 7 ngày trở lại đây (Past 7 days), tháng hiện tại (This month), năm hiện tại (This year).** Sẽ có ích hơn nếu bộ lọc có thêm các khoảng thời gian khác trước đó, bước tiếp theo sẽ bổ sung thêm phần duyệt qua các khoảng ngày tháng như vậy. Lớp **ModelAdmin** có thuộc **date_hierarchy** để cho phép thực hiện điều này.

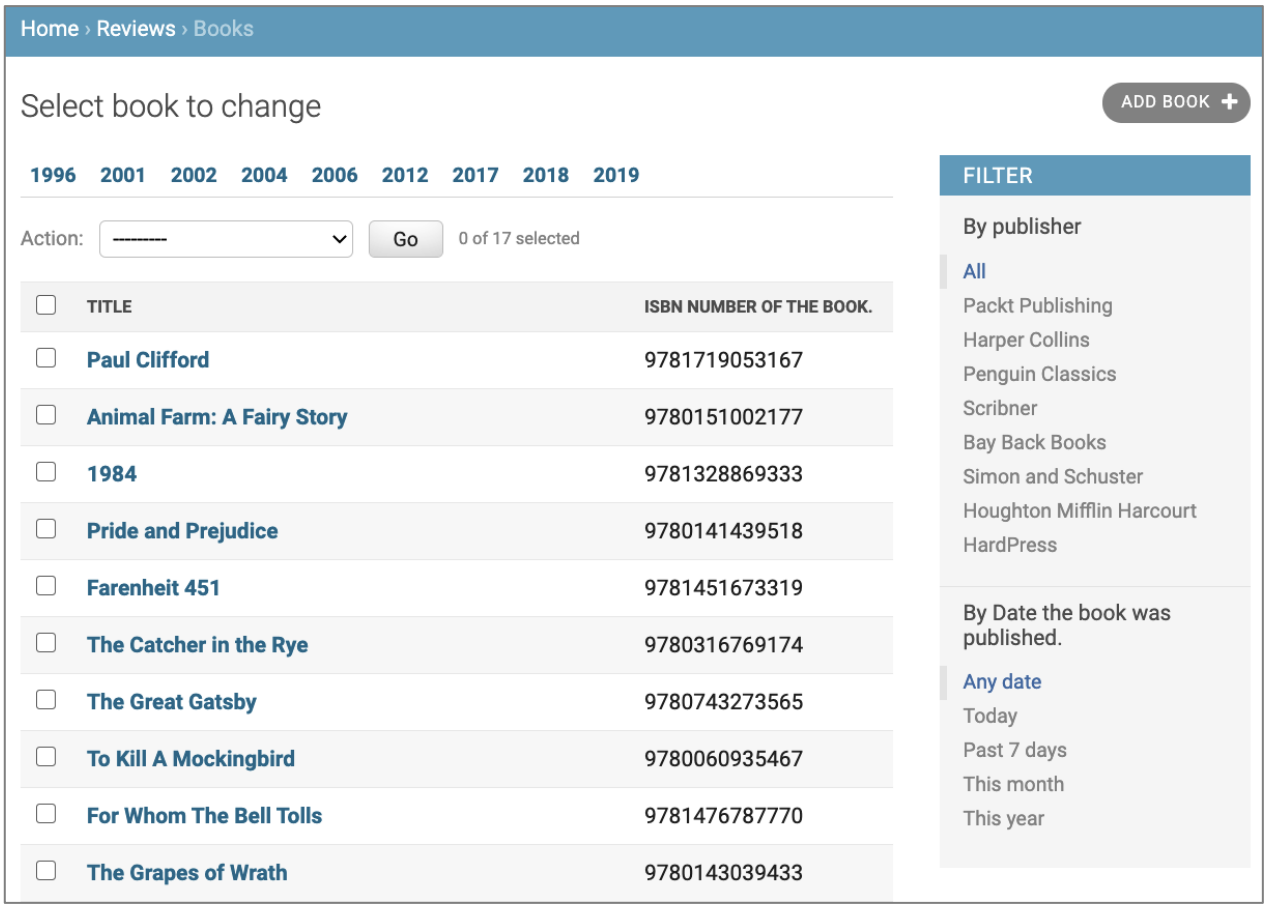


Hình III-6. Các bộ lọc của trang quản lý Book

3. Thêm thuộc tính **date_hierarchy** vào lớp **BookAdmin** và đặt giá trị là **publication_date**.

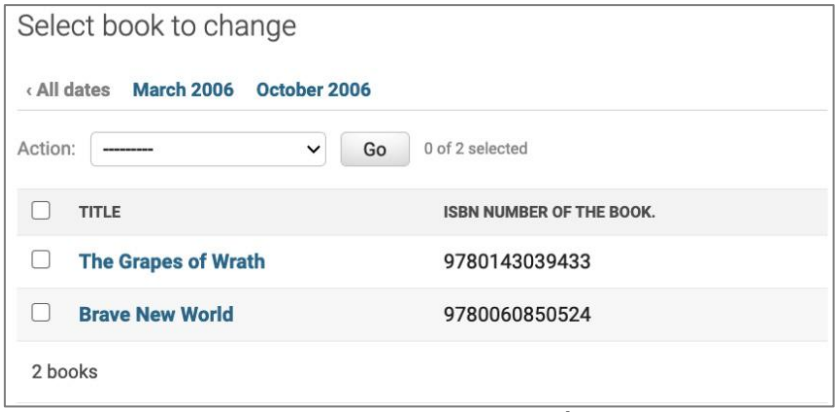
```
class BookAdmin(admin.ModelAdmin):
    date_hierarchy = 'publication_date'
    list_display = ('title', 'isbn13')
    list_filter = ('publisher', 'publication_date')
```

4. Tải lại trang quản lý **Book** và kết quả sẽ như Hình III-7.



Hình III-7. Giao diện trang Danh sách Book với bộ lọc bổ sung về ngày tháng

5. Chọn một năm từ phần bộ lọc ngày tháng ở trên đầu trang (ví dụ năm 2006), kết quả sẽ là trang liệt kê các cuốn sách xuất bản năm 2006 và phần bộ lọc thời gian sẽ cho phép chọn khoảng thời gian chi tiết hơn như Hình III-8.

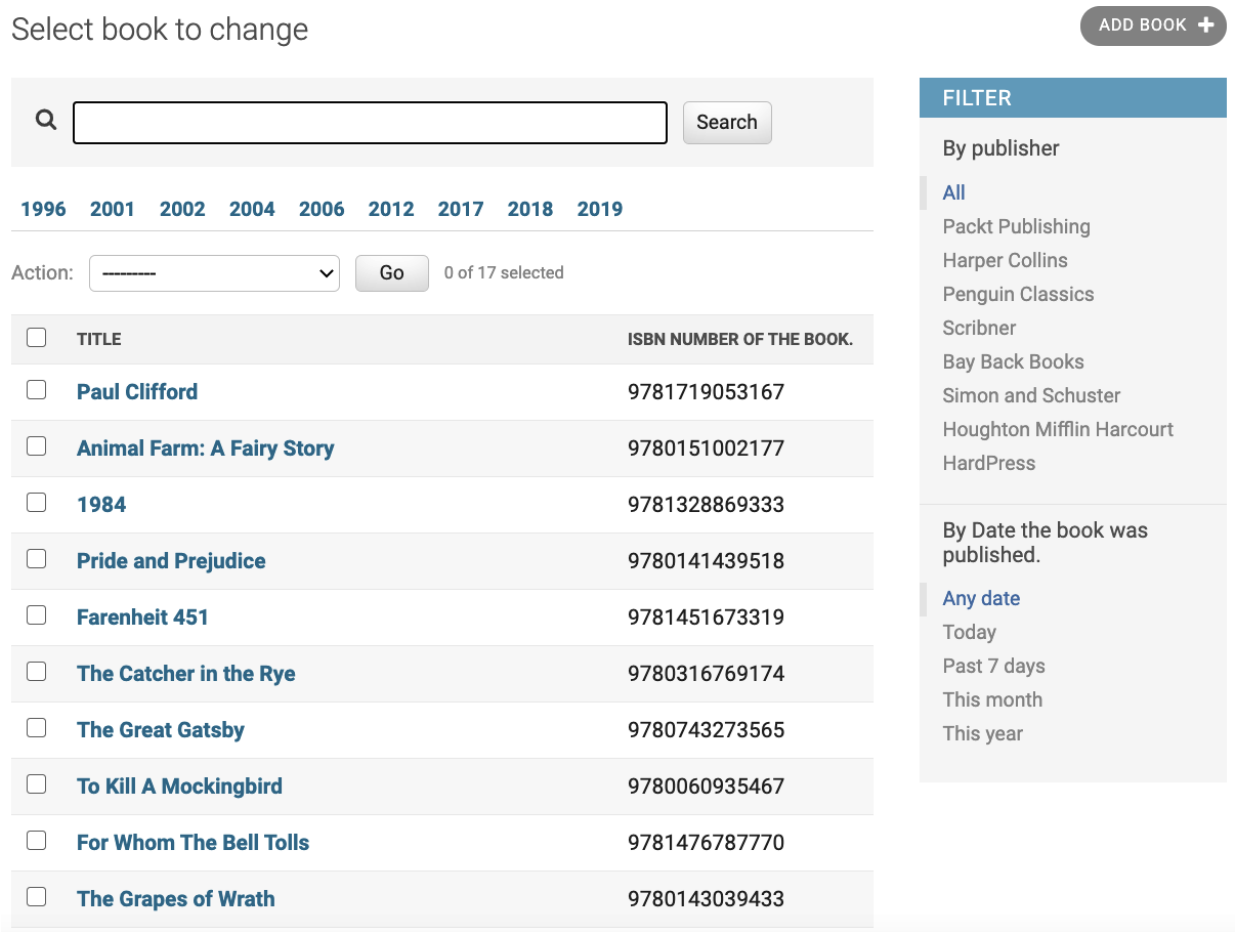


Hình III-8. Trang Danh sách Book xuất bản năm 2006

G. Thêm hộp tìm kiếm

Để bổ sung hộp tìm kiếm sách ở trên đầu trang Danh sách **Book**, ta sử dụng thuộc tính **search_fields** của lớp **ModelAdmin**. Ví dụ, thêm dòng mã lệnh sau vào phần khai báo lớp **BookAdmin** trong tập tin **reviews/admin.py**, kết quả sẽ như Hình III-9.

```
search_fields = ('title', 'isbn')
```



Hình III-9. Trang danh sách Book với hộp tìm kiếm trên đầu

H. Loại trừ và gom nhóm các trường thông tin

Bài này sẽ thực hành loại bỏ trường thông tin **Date edited** khỏi form thêm mới review (**Add review**) và gộp các trường thành 2 nhóm.

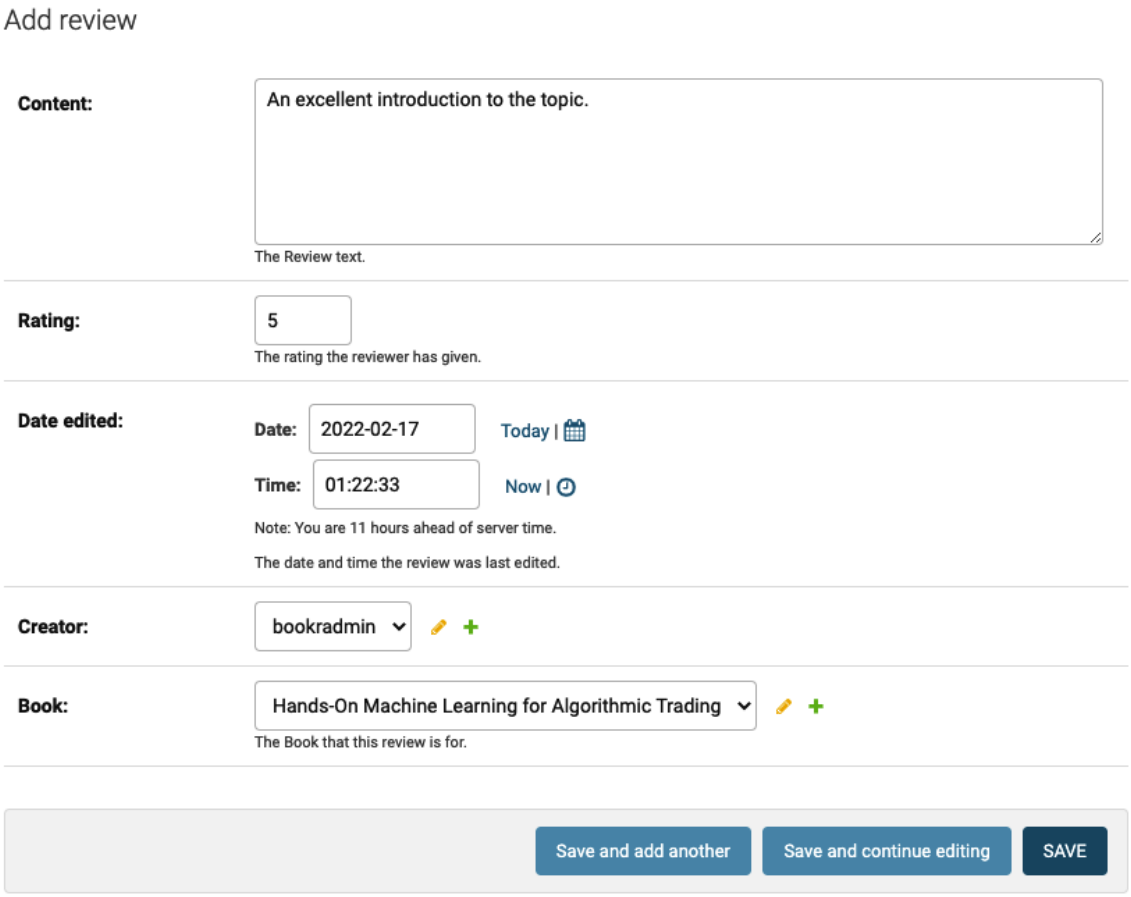
1. Khi mở phần quản lý **Review** và bấm vào nút **ADD REVIEW**, ta có giao diện trang **Add review** như Hình III-10.
2. Ta cần bỏ mục nhập thông tin **Date edited** vì khi tạo mới thì không có thông tin này. Ta cần thêm đoạn mã lệnh sau vào phần định nghĩa lớp **ReviewAdmin** ở tập tin reviews/admin.py:

```
exclude = ('date_edited',)
```

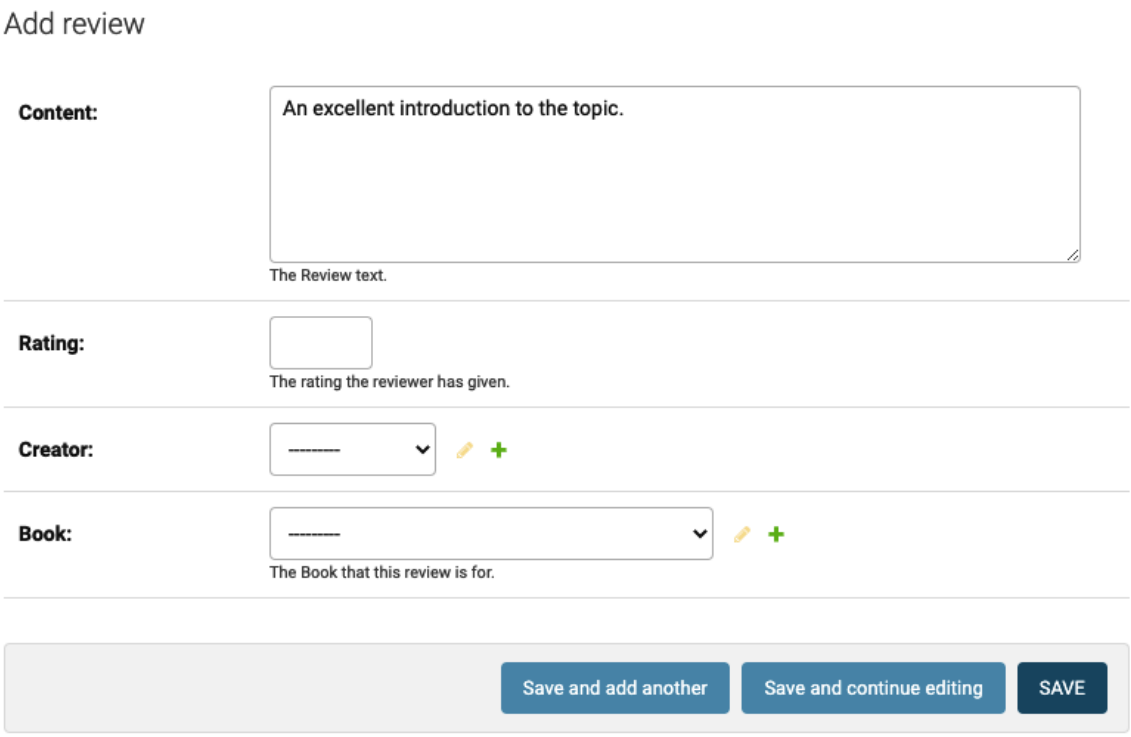
   Kết quả là trang **Add review** sẽ thay đổi thành như Hình III-11.

3. Ta cũng có thể khai báo các trường sẽ xuất hiện trong form nhập thông tin thêm mới **Review** thay vì khai báo theo cách loại bỏ như trên bằng cách sử dụng thuộc tính. Ví dụ, khai báo thuộc tính fields như sau sẽ cho kết quả giống như trong Hình III-11. *(Lưu ý chỉnh lại dòng* ***admin.site.register(Review)*** *thành* ***admin.site.register(Review, ReviewAdmin)****)*

```
fields = ('content', 'rating', 'creator', 'book')
```



Hình III-10. Form nhập thông tin thêm mới Review

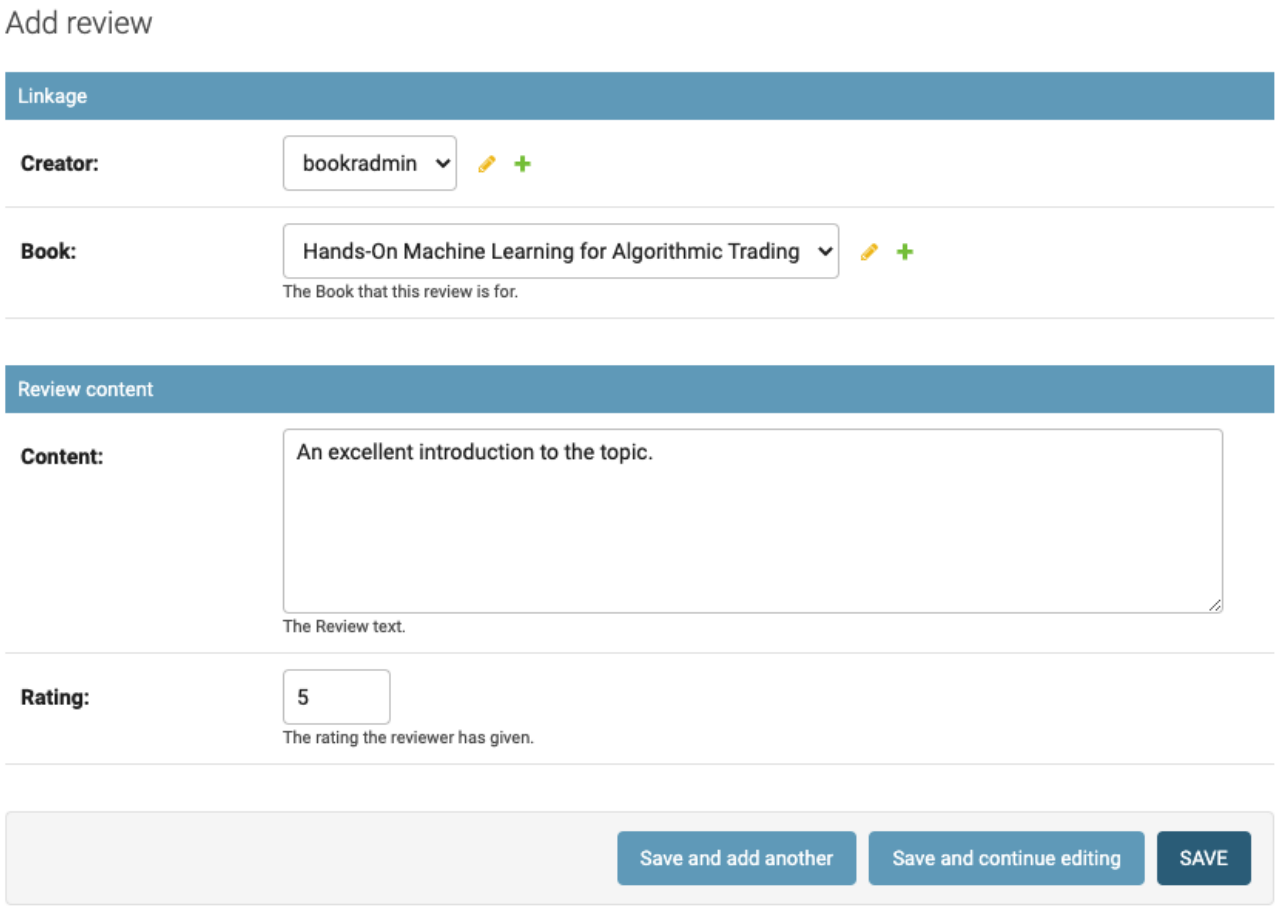


Hình III-11. Form nhập thông tin thêm mới Review sau khi bỏ trường Date edited

4. Để gộp các thuộc tính theo từng nhóm, ta có thể đặt giá trị cho thuộc tính **fieldsets**. Ví dụ khai báo sau sẽ gộp các hộp nhập thông tin cho các trường của bảng **Review** thành **2 nhóm là Linkage và Review content**:

```
fieldsets = (
    ("Linkage", {"fields": ("creator", "book")}),
    ("Review content", {"fields": ("content", "rating")}),
)
```

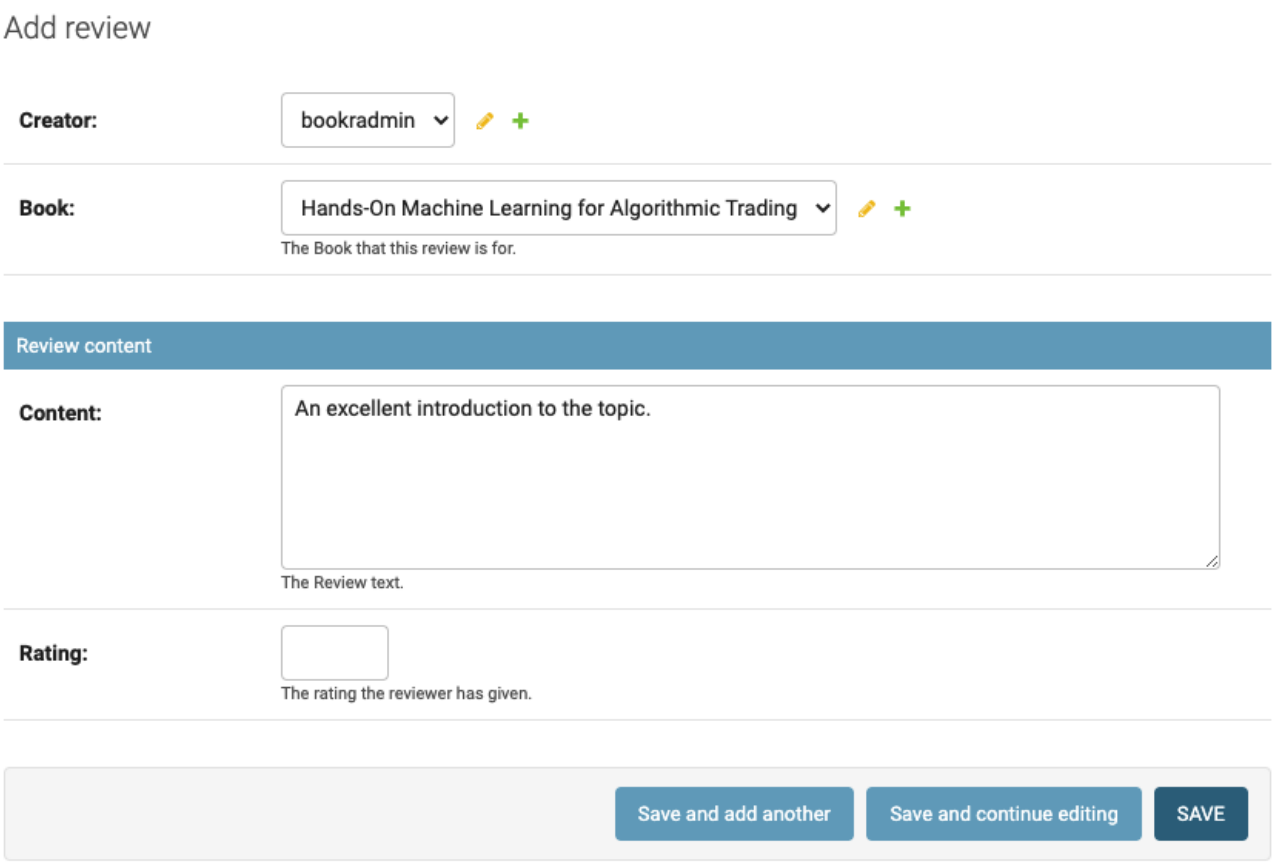
Kết quả sẽ như trong Hình III-12.



Hình III-12. Trang Add review với các trường thông tin được gộp thành 2 nhóm

5. Nếu ta muốn bỏ tiêu đề của từng nhóm trường thì có thể dụng giá trị **None**. Ví dụ đoạn mã sau sẽ tạo ra trang **Add reivew** như Hình III-13.

```
fieldsets = (
    (None, {'fields': ('creator', 'book')}),
    ('Review content', {'fields': ('content','rating')})
)
```



Hình III-13. Trang Add review với nhóm trường đầu tiên không có tiêu đề